CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 35: MCP server integration

> **Exam objective:** Integrate MCP servers into Claude Code and agent applications by selecting the correct server scope, configuring authentication via environment variable expansion, and verifying tool discovery.

Q1.

A development team wants a filesystem MCP server available only within one repository so different projects can use different roots and permissions. Which integration choice best fits this requirement?

A. Register the server globally for the entire workstation B. Configure the server in the project-level Claude or agent configuration for that repository C. Hard-code the server in the application source and commit credentials with it D. Run the server manually for each prompt and let the model infer its tools

Answer: B **Explanation:** Project-level scope is the best choice when server availability should be limited to one repository and travel with that project's configuration. A global registration affects unrelated work, while hard-coding credentials is insecure. Manual launches also make discovery inconsistent and error-prone.

Q2.

An enterprise support team wants a ticketing MCP server available to every developer using Claude Code on a managed laptop image. All users should see the same tool set by default. Which scope is most appropriate?

A. User or machine-wide scope B. Repository-only scope C. Per-prompt scope D. Test-only scope inside CI

Answer: A **Explanation:** A user or machine-wide configuration is appropriate when the same server should be present across many projects and sessions on a managed device. Repository scope is too narrow for this goal. Per-prompt and CI-only approaches do not provide the default interactive availability the scenario requires.

Q3.

You are integrating a cloud API MCP server that requires an access token. Security policy forbids storing secrets directly in the MCP configuration file. Which approach best satisfies the policy?

A. Put the token inline in the JSON configuration but mark the file read-only B. Store the token in an environment variable and reference it through environment variable expansion in the server config

C. Base64-encode the token in the configuration file D. Ask users to paste the token into every request sent to the tool

Answer: B **Explanation:** Environment variable expansion keeps secrets out of source-controlled or shared configuration while still allowing the server process to authenticate. Read-only files and Base64 do not protect the secret meaningfully. Requiring repeated manual pasting is insecure and operationally fragile.

Q4.

After configuring an MCP server, the server process starts successfully, but none of its tools appear in Claude Code. What is the best next verification step?

A. Assume the model will discover the tools after several prompts B. Check the server's advertised tool list or client tool registry to confirm successful tool discovery C. Increase model temperature so tool selection becomes more exploratory D. Rename the server to match the project folder

Answer: B **Explanation:** A running process does not guarantee that the MCP handshake completed or that tools were registered. The correct next step is to inspect tool discovery output, such as logs or the client's tool registry view. Temperature and naming do not fix discovery failures.

Q5.

A team uses separate development and production instances of the same SaaS MCP server. They want the same server definition committed to the repo, while each environment injects different credentials at runtime. Which pattern is best?

A. Keep two committed files, each containing plaintext credentials B. Use one shared server definition with environment variable placeholders resolved differently in each environment C. Disable authentication in development and production D. Tell the model which environment to pretend to use

Answer: B **Explanation:** Using one shared definition plus environment-specific variable values preserves portability while avoiding credential sprawl. Separate plaintext files create maintenance and security problems. Disabling authentication or relying on the model's text alone does not establish a real authenticated connection.

Q6.

An architect is deciding whether to register a database MCP server globally or at the project scope. The database should only be accessible when working on a regulated finance application. What is the strongest reason to choose project scope?

A. Project scope improves model creativity B. Project scope reduces accidental exposure of sensitive tools in unrelated sessions C. Global scope prevents environment variable expansion D. Global scope disables tool discovery caching

Answer: B **Explanation:** Limiting a sensitive server to the relevant project minimizes the chance of accidental invocation in unrelated contexts and aligns with least privilege. Creativity and caching are irrelevant here. Global scope does not inherently disable environment variable expansion.

Q7.

Your MCP server configuration references `${CRM_API_KEY}`, but authentication fails at runtime. The server launches, yet every tool call returns unauthorized. Which root cause is most likely?

A. The environment variable was missing or not available in the process environment used by the client B. The tool descriptions were too verbose C. The model selected the wrong temperature D. The project README lacked setup steps

Answer: A **Explanation:** If the placeholder resolves to an empty or unavailable value, the server can still start but fail authentication on use. Tool description quality and temperature do not explain authorization failures with otherwise reachable tools. Documentation gaps may confuse users but are not the most direct cause.

Q8.

A platform engineer wants to confirm not only that an MCP server is connected, but also that the correct version of the tool schema is being exposed after a deployment. What is the most reliable validation?

A. Verify the server process PID matches yesterday's deployment B. Ask the model if it thinks the tools are current C. Inspect the discovered tool names and input schemas reported to the client after connection D. Confirm the host machine still has internet access

Answer: C **Explanation:** Discovery validation should include both the presence of expected tool names and the schema details the client actually received. A running PID and internet access say nothing about interface correctness. Model opinion is not a substitute for inspecting the registered schema.

Q9.

Two teams share one workstation profile. Team A needs an internal docs MCP server everywhere, but Team B must never see it because they handle external client code on the same machines. What is the best design?

A. Install the server globally and rely on users not to call it B. Configure the server per project or per user profile rather than as a shared global default C. Expose the server globally but hide it from the UI label D. Duplicate the same tool names across both teams' configurations

Answer: B **Explanation:** When visibility must differ between teams, narrower scope such as per-project or separate per-user profiles is safer than a shared global default. Hiding labels does not remove access. Duplicate names increase confusion rather than enforcing separation.

Q10.

An application embeds Claude as an agent and must attach an MCP server only for a single workflow execution using short-lived credentials. Which scoping approach is best?

A. Machine-wide registration B. Repository-wide committed config only C. Runtime or session-scoped attachment within the agent application D. Permanent global installation with a long-lived

token

Answer: C **Explanation:** Session-scoped or runtime attachment matches ephemeral workflows and supports short-lived credentials cleanly. Machine-wide and permanent global approaches keep access available longer than necessary. Repository configuration alone does not ensure per-execution isolation.

Q11.

You added a new MCP server entry and restarted the server process, but Claude Code still shows the old tool set. Which action is most likely required?

A. Reconnect or restart the client session so it performs discovery again B. Increase the model context window C. Change every tool name to a shorter alias D. Remove all environment variables from the configuration

Answer: A **Explanation:** Many clients perform tool discovery during connection initialization, so stale tool lists often mean the client session must reconnect or refresh. Context size and shorter names do not force a new handshake. Removing environment variables would more likely break authentication.

Q12.

A server starts locally from the correct command, and credentials resolve properly, but no tools are discovered. Which integration mistake best explains this?

A. The server failed to implement or expose the MCP capability negotiation and tool listing correctly B. The server is using HTTPS instead of HTTP C. The objective text was copied incorrectly into documentation D. The model was asked a simple question before any tool task

Answer: A **Explanation:** Tool discovery depends on successful protocol-level capability negotiation and tool exposure, not merely process startup. Transport details may matter in some implementations, but the core symptom points to a protocol or capability exposure issue. Documentation and prompt order are unrelated.

Q13.

An engineer wants to verify that environment variable expansion is working without revealing the actual secret value in logs. Which method is best?

A. Print the entire resolved token to the console for comparison B. Validate through a successful authenticated tool discovery or lightweight authenticated call, while masking secrets in logs C. Commit the secret temporarily and remove it later D. Ask the model to guess whether the token was loaded

Answer: B **Explanation:** The safest verification approach is behavioral: confirm authenticated operation without exposing secret material. Printing or committing secrets violates security practice. The model cannot reliably infer whether secure expansion occurred without system evidence.

Q14.

A multi-repo monorepo setup includes a payments service and a marketing site. Only the payments area should expose a ledger MCP server. What is the best configuration strategy?

A. Add the server to the root global configuration for convenience B. Scope the server to the payments project configuration rather than the entire monorepo environment C. Embed credentials directly in the marketing site's documentation D. Register the ledger server under the same name as a public calculator tool

Answer: B **Explanation:** Narrow project scoping prevents sensitive tooling from appearing in unrelated areas of the monorepo. Root or global convenience conflicts with least privilege. Misleading names and credential leakage create both security and usability problems.

Q15.

During rollout, a team wants a final acceptance check proving an MCP integration is ready for users. Which checklist item is most complete?

A. The server binary exists on disk B. The server process launches with no syntax errors C. The client discovers the expected tools, authentication succeeds through environment variables, and a representative tool call completes successfully D. The model mentions the server in its chain-of-thought

Answer: C **Explanation:** A production-ready validation must confirm the full path: configuration, auth resolution, tool discovery, and successful invocation. Existence of binaries or clean startup alone is insufficient. Internal model reasoning is neither observable nor a deployment criterion.